FINAL ML PROJECT

By: Astrid Lorenzana (ITAI-1317)

FIRST 6 MODELS

Following my group's MIDTERM project, based off the dataset chosen, six models had to be trained, evaluated, and show results for how well they performed in terms of accuracy, precision, recall, f1_score, and roc-auc score.

```
Training Logistic Regression
Training Decision Tree
Training Random Forest
Training Gradient Boosting
Training KNN
Training Support Vector Machine
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC_AUC
2	Random Forest	0.774621	0.575000	0.575000	0.575000	0.826420
3	Gradient Boosting	0.750000	0.517699	0.835714	0.639344	0.845501
0	Logistic Regression	0.735795	0.501099	0.814286	0.620408	0.844567
5	Support Vector Machine	0.734848	0.500000	0.825000	0.622642	0.831878
1	Decision Tree	0.722538	0.477193	0.485714	0.481416	0.646189
4	KNN	0.682765	0.440860	0.732143	0.550336	0.767905

Based off each model's results, I chose that the best performing models were random forest, gradient boosting, and logistic regression.

VOTING VS AVERAGE

```
# ============================================================
# Voting Classifier
# ============================================================
voting_model = VotingClassifier(estimators=[("rf", RandomForestClassifier(random_st
                                ("gb", GradientBoostingClassifier(random_state=42))
                                ("lr", LogisticRegression(max_iter=1000, random_sta
                                voting="soft")
```

```
# ============================================================
# Average of predicted probabilities on validation set
# ============================================================
rf_proba = models["Random Forest"].predict_proba(X_validation)[:,1]
gb_proba = models["Gradient Boosting"].predict_proba(X_validation)[:,1]
lr_proba = models["Logistic Regression"].predict_proba(X_validation)[:,1]

average_proba = (rf_proba + gb_proba + lr_proba)/3
average_predictions = np.where(average_proba >=0.50, "Yes", "No")
```

The images on the side show how the models were stored properly in the respective models, voting classifier and average ensemble models, so that they could be used in the program through the metrics required to prove their performance.

The models are ensembled to provide better results. This shows how “wisdom of the crowd” works because by each model providing their input on what the result could be, we can put their responses together to give us an estimate with higher chances of being correct.

VOTING VS BAYESIAN

	Model	Accuracy	Precision	Recall	F1 Score	ROC_AUC
0	Voting Classifier	0.774834	0.558583	0.729537	0.632716	0.839922
1	Average Predictions	0.774834	0.558583	0.729537	0.632716	0.839922

Through comparing the voting classifier model and the average ensemble models, the outcomes displayed equal results. However, since I needed one of those models to compare it to the Bayesian model, I went with the voting classifier ensemble model. (The image above is based off the test dataset.)

	Model	Accuracy	Precision	F1 Score	Recall	ROC_AUC
0	Voting Classfier	0.756629	0.527981	0.628075	0.775000	0.846827
1	Gaussian Naive Bayes	0.698864	0.462891	0.598485	0.846429	0.803652

Once I established the code to train and evaluate the Bayesian model, I compared it to the model of my choice and found that the voting classifier model performed better in most metrics; the exception being recall. The Bayesian model had a better recall than the voting classifier model.